

main.cpp	Corre ▶	Produção
<pre>1 #include <iostream> 2 using namespace std; //questão 9 3 4 int main() { 5 int v[10]; 6 for (int i = 0; i < 10; i++) { 7 cout << "Digite o " << i + 1 << "º numero: "; 8 cin >> v[i]; 9 } 10 cout << "\nValores armazenados: "; 11 for (int i = 0; i < 10; i++) { 12 cout << v[i] << " "; 13 } 14 return 0; 15 }</pre>		Digite o 1º numero: 1 Digite o 2º numero: 2 Digite o 3º numero: 3 Digite o 4º numero: 4 Digite o 5º numero: 5 Digite o 6º numero: 67 Digite o 7º numero: 76 Digite o 8º numero: 8 Digite o 9º numero: 9 Digite o 10º numero: 12 Valores armazenados: 1 2 3 4 5 67 76 8 9 12

main.cpp	Corre ▶	Produção
<pre>1 #include <iostream> 2 using namespace std; //questão 10 3 4 int main() { 5 int v[10]; 6 for (int i = 0; i < 10; i++) { 7 cout << "Digite o " << i + 1 << "º numero: "; 8 cin >> v[i]; 9 } 10 int maior = v[0], menor = v[0]; 11 int posMaior = 0, posMenor = 0; 12 13 for (int i = 1; i < 10; i++) { 14 if (v[i] > maior) { maior = v[i]; posMaior = i; } 15 if (v[i] < menor) { menor = v[i]; posMenor = i; } 16 } 17 18 cout << "\nMaior: " << maior << " (Indice " << posMaior << ")"; 19 cout << "\nMenor: " << menor << " (Indice " << posMenor << ")\n"; 20 return 0; 21 }</pre>		Digite o 1º numero: 1 Digite o 2º numero: 2 Digite o 3º numero: 3 Digite o 4º numero: 4 Digite o 5º numero: 5 Digite o 6º numero: 9 Digite o 7º numero: 14 Digite o 8º numero: 24 Digite o 9º numero: 34 Digite o 10º numero: 55 Maior: 55 (Indice 9) Menor: 1 (Indice 0)

main.cpp	Corre ▶	Produção
<pre>1 #include <iostream> 2 using namespace std; //questão 11 3 4 int main() { 5 float notas[5], soma = 0; 6 for (int i = 0; i < 5; i++) { 7 cout << "Nota do aluno " << i + 1 << "º: "; 8 cin >> notas[i]; 9 soma += notas[i]; 10 } 11 float media = soma / 5; 12 float maior = notas[0], menor = notas[0]; 13 int acimaMedia = 0; 14 15 for (int i = 0; i < 5; i++) { 16 if (notas[i] > maior) maior = notas[i]; 17 if (notas[i] < menor) menor = notas[i]; 18 if (notas[i] > media) acimaMedia++; 19 } 20 21 cout << "\nMedia da turma: " << media; 22 cout << "\nMaior nota: " << maior; 23 cout << "\nMenor nota: " << menor; 24 cout << "\nNotas acima da media: " << acimaMedia << "\n"; 25 return 0; 26 }</pre>		Nota do aluno 1: 6 Nota do aluno 2: 3 Nota do aluno 3: 10 Nota do aluno 4: 7 Nota do aluno 5: 8 Media da turma: 6.8 Maior nota: 10 Menor nota: 3 Notas acima da media: 3

main.cpp

Corre ▶

Produção

```

1 #include <iostream>
2 using namespace std; // questão 12
3
4 int main() {
5     int v[10], pares = 0, impares = 0;
6     for (int i = 0; i < 10; i++) {
7         cout << "Numero " << i + 1 << ": ";
8         cin >> v[i];
9         if (v[i] % 2 == 0) pares++;
10        else impares++;
11    }
12    cout << "\nQuantidade de pares: " << pares;
13    cout << "\nQuantidade de impares: " << impares << "\n";
14    return 0;
15 }

```

Numero 1: 33
 Numero 2: 24
 Numero 3: 90
 Numero 4: 77
 Numero 5: 85
 Numero 6: 1957
 Numero 7: 3465
 Numero 8: 88795
 Numero 9: 1
 Numero 10: 95746463839929028263545545

Quantidade de pares: 2
 Quantidade de impares: 8

main.cpp

Corre ▶

Produção

```

1 #include <iostream>
2 using namespace std; // questão 13
3
4 int main() {
5     int v[10], busca, pos = -1;
6     for (int i = 0; i < 10; i++) {
7         cout << "Digite o " << i + 1 << "º numero: ";
8         cin >> v[i];
9     }
10    cout << "Digite o valor para buscar: ";
11    cin >> busca;
12
13    for (int i = 0; i < 10; i++) {
14        if (v[i] == busca) {
15            pos = i;
16            break;
17        }
18    }
19
20    if (pos != -1) cout << "Valor encontrado na posicao (indice): " << pos
21    << "\n";
22    else cout << "Valor nao encontrado\n";
23    return 0;
24 }

```

Digite o 1º numero: 7357
 Digite o 2º numero: 1
 Digite o 3º numero: 1346
 Digite o 4º numero: 2
 Digite o 5º numero: 1406
 Digite o 6º numero: 3
 Digite o 7º numero: 2109
 Digite o 8º numero: 4
 Digite o 9º numero: 1234
 Digite o 10º numero: 5
 Digite o valor para buscar: 1234
 Valor encontrado na posicao (indice): 8

```

1 struct No {
2     int valor;
3     No* proximo;
4 };
5
6 void inserirInicio(No*& inicio, int valor) {
7     No* novo = new No();
8     novo->valor = valor;
9     novo->proximo = inicio;
10    inicio = novo;
11 }

```

questão 17

```
main.cpp

1 // questão 18
2 void exibirLista(No* inicio) {
3     No* atual = inicio;
4     while (atual != nullptr) {
5         cout << atual->valor << " -> ";
6         atual = atual->proximo;
7     }
8     cout << "NULL\n";
9 }
```

```
main.cpp

1 // questão 19
2 bool buscar(No* inicio, int valor) {
3     No* atual = inicio;
4     while (atual != nullptr) {
5         if (atual->valor == valor) return true;
6         atual = atual->proximo;
7     }
8     return false;
9 }
```

main.cpp

Corre ▶

Produção

```
1 #include <iostream>
2 using namespace std; //questão 20
3
4 int main() {
5     float notas[10], soma = 0;
6     for (int i = 0; i < 10; i++) {
7         cout << "Nota do aluno " << i + 1 << ": ";
8         cin >> notas[i];
9         soma += notas[i];
10    }
11
12    float media = soma / 10;
13    float maior = notas[0], menor = notas[0];
14    int acimaMedia = 0;
15
16    for (int i = 0; i < 10; i++) {
17        if (notas[i] > maior) maior = notas[i];
18        if (notas[i] < menor) menor = notas[i];
19        if (notas[i] > media) acimaMedia++;
20    }
21
22    cout << "\n--- RELATORIO ---";
23    cout << "\nNotas: ";
24    for (int i = 0; i < 10; i++) cout << notas[i] << " ";
25    cout << "\nMedia: " << media;
26    cout << "\nMaior Nota: " << maior;
27    cout << "\nMenor Nota: " << menor;
28    cout << "\nAlunos acima da media: " << acimaMedia << "\n";
29
30    float busca;
31    cout << "\nDigite uma nota para procurar: ";
32    cin >> busca;
33    bool achou = false;
34    for (int i = 0; i < 10; i++) {
35        if (notas[i] == busca) {
36            cout << "Nota encontrada na posicao " << i << "\n";
37            achou = true;
38            break;
39        }
40    }
41    if (!achou) cout << "Nota nao encontrada.\n";
42
43    return 0;
44 }
```

Nota do aluno 1: 6
Nota do aluno 2: 5
Nota do aluno 3: 10
Nota do aluno 4: 9
Nota do aluno 5: 3
Nota do aluno 6: 5
Nota do aluno 7: 2
Nota do aluno 8: 8
Nota do aluno 9: 10
Nota do aluno 10: 9

--- RELATORIO ---
Notas: 6 5 10 9 3 5 2 8 10 9
Media: 6.7
Maior Nota: 10
Menor Nota: 2
Alunos acima da media: 5

Digite uma nota para procurar: 10
Nota encontrada na posicao 2

main.cpp

Corre

Produção

```

1 #include <iostream>
2 using namespace std;
3
4 struct No {
5     int valor;
6     No* proximo;
7 };
8
9 void inserir(No* inicio, int val) {
10     No* novo = new No(val, inicio);
11     inicio = novo;
12 }
13
14 void listar(No* inicio) {
15     No* p = inicio;
16     while (p) { cout << p->valor << " -> "; p = p->proximo; }
17     cout << "NULL\n";
18 }
19
20 bool buscar(No* inicio, int val) {
21     No* p = inicio;
22     while (p) {
23         if (p->valor == val) return true;
24         p = p->proximo;
25     }
26     return false;
27 }
28
29 void remover(No* inicio, int val) {
30     No* p = inicio;
31     No* ant = nullptr;
32     while (p && p->valor != val) {
33         ant = p;
34         p = p->proximo;
35     }
36     if (!p) return;
37     if (!ant) inicio = p->proximo;
38     else ant->proximo = p->proximo;
39     delete p;
40 }
41
42 int contar(No* inicio) {
43     int c = 0;
44     for (No* p = inicio; p; p = p->proximo) c++;
45     return c;
46 }
47
48 int main() {
49     No* inicio = nullptr;
50     int op, val;
51
52     do {
53         cout << "1-Insérir 2-Listar 3-Buscar 4-Remover 5-Contar 0-Sair\nOpcao: ";
54         cin >> op;
55         if (op == 1) { cout << "Valor: "; cin >> val; inserir(inicio, val); }
56         else if (op == 2) listar(inicio);
57         else if (op == 3) { cout << "Buscar: "; cin >> val; cout << (buscar(inicio, val) ? "Encontrado\n" : "Não encontrado\n"); }
58         else if (op == 4) { cout << "Remover: "; cin >> val; remover(inicio, val); }
59         else if (op == 5) cout << "Total de elementos: " << contar(inicio) << "\n";
60     } while (op != 0);
61
62     return 0;
63 }

```

1-Insérir 2-Listar 3-Buscar 4-Remover 5-Contar 0-Sair
Opcao: 1
Valor: 50
1-Insérir 2-Listar 3-Buscar 4-Remover 5-Contar 0-Sair
Opcao: 2
50 -> NULL
1-Insérir 2-Listar 3-Buscar 4-Remover 5-Contar 0-Sair
Opcao: 3
Buscar: 50
Encontrado
1-Insérir 2-Listar 3-Buscar 4-Remover 5-Contar 0-Sair
Opcao: 4
Remover: 8
1-Insérir 2-Listar 3-Buscar 4-Remover 5-Contar 0-Sair
Opcao: 5
Total de elementos: 1
1-Insérir 2-Listar 3-Buscar 4-Remover 5-Contar 0-Sair
Opcao: 0

questão 21

Questão 21

A lista encadeada é a mais adequada. Como a quantidade de registros muda constantemente. A lista permite alterar e deslocar memória de forma plimônica sem custo de reorganização/deslocamento de elementos na memória.

CARAC.	VETOR	LISTA ENCADEADA
TAM.	Fixo	Dinâmica
ACES.	Rápido e direto por índice	Sequencial, só por nó
INSER. REMOV.	Lento	Rápido
MEM.	eficiente / contíguo	Possui custo extra para armazenar os ponteiros
Exemplos	Armazenar notas fixas de tur.	Fila de atendimento ou histórico